CSE2142: Git Workflow & Viva Complete Guide

Part 1: Git Project Workflow (Step-by-Step)

Step 1: Git Initial Setup (একবারই করতে হবে)

পিসিতে গিট ইন্সটল করার পর আপনার পরিচয় সেট করতে হবে, যাতে কমিটগুলোর ট্র্যাক থাকে।

```
git config --global user.name "Your Name" git config --global user.email "your.email@example.com"
```

Step 2: Clone the Repository

গিটহাব থেকে স্যারের দেওয়া প্রজেক্টটি পিসিতে নামাতে হবে এবং সেই ফোল্ডারে প্রবেশ করতে হবে।

```
git clone https://github.com/m-r-kushal/CSE2142-2026-G1.git cd CSE2142-2026-G1
```

Step 3: Task-by-Task Execution (Copy-Paste Guide)

Task 0: Initial Check & Command Usage

প্রোগ্রামটি রান করার জন্য আর্গুমেন্ট পাস করতে হয়। নিচে সবগুলোর ব্যবহার দেওয়া হলো:

```
javac StudentList.java // 1. l = list (সব এমপ্লয়ি দেখার জন্য) java StudentList l // 2. s = show (র‍্যান্ডম ১ জনকে দেখার জন্য) java StudentList s // 3. + = add (নতুন এমপ্লয়ি যুক্ত করার জন্য) java StudentList +NewEmployee // 4. ? = search (কাউকে খোঁজার জন্য) // (cmd তে ? কাজ না করলে "?" ব্যবহার করবেন) java StudentList ?John // 5. c = count (মোট কতজন আছে তা গোনার জন্য) java StudentList c // 6. u = update (ফাইল আপডেট করার জন্য) java StudentList u
```

Task 1: Update Code Style

Code: আপনার IDE (যেমন VS Code/IntelliJ) তে ফাইলটি ওপেন করে 'Format Document' করুন। সব Braces {} এবং Indentation ঠিক করুন।

```
git checkout -b task-1 git add . git commit -m "Task #1. Update Code Style for Better Consistency:" git checkout master git merge task-1
```

Task 2: Terminate Early on Wrong Args

Code: `main` মেথডের একদম শুরুতে এই কোডটি বসান:

```
if (args.length != 1) { System.out.println("Error: Wrong number of arguments!"); return; }
```

```
git checkout -b task-2 git add . git commit -m "Task #2. Application Now Terminates Early if the Number of Arguments Passed into It Is Wrong, Fix It:" git checkout master git merge task-2
```

Task 3: Meaningful Variable Names

Code: `StudentList.java` ফাইলে থাকা ছোট ভেরিয়েবল (যেমন: `s`, `i`) গুলোকে রিনেম (Shift+F6) করে `line`, `index`, `fileReader`, `employeeName` এর মতো নাম দিন।

```
git checkout -b task-3 git add . git commit -m "Task #3. Makes Improvements to Variable Names:" git checkout master git merge task-3
```

Task 4: Refactor File Read/Write

Code: ক্লাসের ভেতরে নিচের দুটি মেথড তৈরি করুন এবং মেইন মেথড থেকে এগুলো কল করুন:

```
public static String readFromFile(String fileName) { try { BufferedReader reader = new BufferedReader( new InputStreamReader(new FileInputStream(fileName))); return reader.readLine(); } catch (Exception e) { return ""; } } public static void writeToFile(String fileName, String text) { try { BufferedWriter writer = new BufferedWriter( new FileWriter(fileName, true)); // true for append writer.write(", " + text); writer.close(); } catch (Exception e) {} }
```

```
git checkout -b task-4 git add . git commit -m "Task #4. Refactors Duplicate File Read and Write Logic into Methods:" git checkout master git merge task-4
```

Task 5: Constants Class

Code: একই ফোল্ডারে `Constants.java` নামে নতুন ফাইল খুলুন:

```
public class Constants { public static final String FILE_NAME = "employees.txt"; public static final String DATE_FORMAT = "dd/mm/yyyy-hh:mm:ss a"; }
```

এরপর `StudentList.java` ফাইলে ফাইলের নামের জায়গায় `Constants.FILE_NAME` বসান।

```
git checkout -b task-5 git add . git commit -m "Task #5. Replaces String Literals with Constants, Storing Those Constants in a New Class Called Constants.java:" git checkout master git merge task-5
```

Task 6: Remove Temporary Variables

Code: কোডে যদি এমন কোনো `String temp = "";` থাকে যা পরবর্তীতে কোনো কাজেই লাগেনি, সেগুলো মুছে দিন।

```
git checkout -b task-6 git add . git commit -m "Task #6. Remove Temporary Variables:" git checkout master git merge task-6
```

Task 7: Eliminate 'done' Variable

Code: সার্চ অপশনের (?) লজিকটি এমনভাবে লিখুন:

```
String searchName = args[0].substring(1);
String[] employees = fileContent.split(", "); for
(String emp : employees) { if
(emp.trim().equals(searchName)) {
System.out.println("We found it!"); return; //
Exits directly, no 'done' variable needed } }
System.out.println("Not found!");
```

```
git checkout -b task-7 git add . git commit -m
"Task #7. Eliminates the 'done' Control-Flow
Variable. Adds Better Response for Search
Operation:" git checkout master git merge task-7
```

Task 8: Simplify Count Operation

Code: `c` আর্গুমেন্টের জন্য ক্যারেক্টার লুপ না চালিয়ে সহজভাবে লিখুন:

```
String fileContent =
readFromFile(Constants.FILE_NAME); String[]
employees = fileContent.split(", ");
System.out.println(employees.length + " word(s)
found ");
```

```
git checkout -b task-8 git add . git commit -m
"Task #8. Simplifies the Logic Behind the Count
Operation:" git checkout master git merge task-8
```

Task 9: Handle Invalid Arguments

Code: `if-else` বা `switch` এর শেষে ডিফল্ট কেস দিন:

```
else { System.out.println("Error: Invalid
argument passed! Use l, s, +, ?, c, or u."); }
```

```
git checkout -b task-9 git add . git commit -m
"Task #9. Adds Handling for Case When User Enters
Invalid Arguments:" git checkout master git merge
task-9
```

Task 10: Comments & Naming

Code: ক্লাসের শুরুতে এবং মেথডগুলোর উপরে কমেন্ট দিন। যেমন:

```
// This class manages the employee list
operations // such as read, write, search and
count.
```

```
git checkout -b task-10 git add . git commit -m
"Task #10. Add More Comments and Makes More
Naming Improvements:" git checkout master git
merge task-10
```

Step 4: Push to Github

সব কাজ শেষ হয়ে গেলে ফাইনাল কোড রিমোট রিপোজিটরিতে পুশ করুন:

```
git push origin master
```

Part 2: Lab Viva Questions & Answers

Q1: What is Git and why do we use it?

Ans: Git হলো একটি ডিস্ট্রিবিউটেড ভার্সন কন্ট্রোল সিস্টেম। কোডের পরিবর্তন ট্র্যাক করা, বিভিন্ন ভার্সন মেইনটেইন করা এবং একাধিক মানুষের একসাথে একটি প্রজেক্টে কাজ করার সুবিধার জন্য এটি ব্যবহার করা হয়।

Q2: Difference between `git add` and `git commit`?

Ans: `git add` কমান্ডটি কোডের পরিবর্তনগুলোকে "Staging area"-তে পাঠায় (অর্থাৎ সেভ করার জন্য প্রস্তুত করে)। আর `git commit` কমান্ডটি স্টেজ করা ফাইলগুলোকে একটি মেসেজ সহ লোকাল রিপোজিটরিতে পার্মানেন্টলি সেভ করে।

Q3: Why do we use branching in Git?

Ans: মূল কোডকে (master branch) অক্ষত রেখে নতুন কোনো ফিচার যুক্ত করা বা বাগ ফিক্স করার জন্য ব্রাঞ্চ ব্যবহার করা হয়। কাজ সফল হলে সেটি মূল কোডে মার্জ করা যায়।

Q4: What is the purpose of `git checkout -b`?

Ans: এটি একসাথে দুটি কাজ করে: একটি নতুন ব্রাঞ্চ তৈরি করে এবং সাথে সাথে নতুন তৈরি করা সেই ব্রাঞ্চে সুইচ করে।

Q5: What happens when you do `git merge`?

Ans: এটি একটি ব্রাঞ্চের (যেমন: task-1) পরিবর্তনগুলোকে অন্য একটি ব্রাঞ্চের (যেমন: master) সাথে যুক্ত বা কম্বাইন করে দেয়।

Q6: Difference between `git clone` and `git pull`?

Ans: `git clone` প্রথমবারের মতো রিমোট রিপোজিটরি থেকে পুরো প্রজেক্ট পিসিতে ডাউনলোড করে। আর `git pull` এক্সিস্টিং লোকাল রিপোজিটরিতে রিমোট রিপোজিটরি থেকে নতুন কোনো আপডেট আসলে তা ফেচ ও মার্জ করে।

Q7: What does `git push` do?

Ans: এটি পিসির লোকাল রিপোজিটরির সমস্ত কমিট বা পরিবর্তনগুলোকে গিটহাবের রিমোট রিপোজিটরিতে আপলোড করে।

Q8: (From Java Task 2) Why is argument length checking necessary?

Ans: প্রোগ্রাম রান করার সময় ইউজার যদি কোনো আর্গুমেন্ট না দেয়, তাহলে কোড `args[0]` এক্সেস করতে গিয়ে `ArrayIndexOutOfBoundsException` খাবে। এটি যেন না হয় তাই `args.length` চেক করা হয়।

Q9: (From Java Task 4) What is Refactoring?

Ans: রিফ্যাক্টরিং হলো কোডের বাহ্যিক আউটপুট বা ব্যবহার পরিবর্তন না করে, কোডের ভিতরের স্ট্রাকচার উন্নত বা পরিষ্কার করার প্রক্রিয়া। যেমন: ডুপ্লিকেট কোডকে মেথডে রূপান্তর করা।

Q10: What is a constant class (Task 5)?

Ans: এটি এমন একটি ক্লাস যেখানে প্রোগ্রামের ফিক্সড ডেটাগুলো (যেমন: ফাইলের নাম "employees.txt") `public static final` ভেরিয়েবল হিসেবে রাখা হয়, যেন যেকোনো জায়গা থেকে সহজে ব্যবহার ও ম্যানেজ করা যায়।